Farmacia Hospitalaria

Resumen de la carpeta de estudio

Sistema de Gestión de Stock — Farmacia Hospitalaria

Lo esencial de los 18 capítulos de la carpeta de estudio, para repasar antes de la defensa.

Trabajo Final Integrador — Tecnicatura Universitaria en Programación — UTN
Noviembre de 2026

Índice

1. El proyecto en un vistazo 3
2. Repaso de programación 4
3. Cómo funciona la web 4
4. Bases de datos y SQL 5
5. Java 6
6. Spring Boot 7
7. Recorrido del backend 8
8. Seguridad 8
9. Reglas de negocio: FEFO, stock y concurrencia 9
10. JavaScript y TypeScript 11
11. React 11
12. Recorrido del frontend 11
13. Docker y entornos 12
14. Pruebas 12
15. Git y herramientas 13
16. Checklist antes de la defensa 13

Resumen de la carpeta de estudio

Este resumen junta lo esencial de los 18 capítulos de la carpeta de estudio. Sirve para repasar rápido antes de la defensa. Para el detalle y el código completo, está la carpeta de estudio.

> **Para la defensa:** "Es un sistema web de gestión de stock por lote para la farmacia de un hospital público. Registra ingresos y transferencias a las áreas aplicando FEFO automáticamente, avisa vencimientos y faltantes, deja trazabilidad de cada movimiento y genera reportes. Está hecho con un backend en Java con Spring Boot, una base PostgreSQL normalizada y un frontend en React, con seguridad por roles y pruebas automáticas."

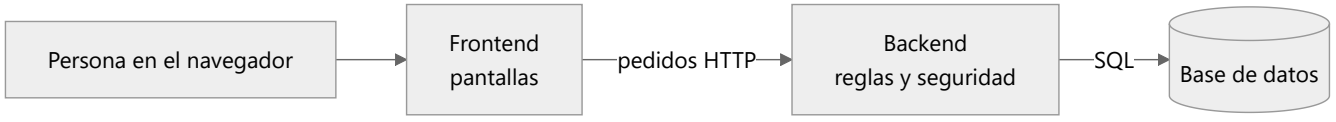
1. El proyecto en un vistazo

La farmacia del hospital **recibe** mercadería de las droguerías (cada caja con número de lote y vencimiento) y **entrega** medicamentos a las áreas (Guardia, Terapia, Pediatría…). Sin control, los medicamentos se vencen en el estante o faltan cuando se necesitan. Y si un laboratorio retira un lote, hay que saber dónde está cada unidad.

El sistema permite registrar ingresos y transferencias, avisar vencimientos y faltantes, dar de baja lo vencido, corregir diferencias de inventario y sacar reportes en Excel y PDF. Todo con usuario, contraseña y permisos por rol.

Las piezas

Pieza	Qué hace	Tecnología
Frontend	Las pantallas en el navegador.	React 19 + TypeScript + Vite
HTTP	Los mensajes entre navegador y servidor.	JSON
Backend	Aplica las reglas y la seguridad.	Java 21 + Spring Boot 4.1
Base de datos	Guarda todo ordenado.	PostgreSQL 16 + Flyway
Contenedores	Levantan todo con un comando.	Docker



Regla de oro: el frontend **muestra** y el backend **decide**. Esconder un botón es comodidad; la seguridad real está en el backend.

Roles

Rol	Quién	Qué puede hacer
ADMIN	Jefe/a de farmacia	Todo, incluido crear usuarios.
FARMACEUTICO	Farmacéutico/a o técnico/a	Ingresos, transferencias, bajas, ajustes y catálogo.
CONSULTA	Dirección, auditoría	Solo mirar y sacar reportes.

Las 10 reglas para saber de memoria

1. **Vencido** = vence **antes de hoy**. El día del vencimiento todavía se usa.
2. **Por vencer** = vence entre hoy y dentro de 30 días.
3. **Stock total** = suma de los lotes **no vencidos**. No se guarda: se calcula.
4. **Sin stock** = 0. **Stock bajo** = más de 0 pero ≤ stock mínimo.
5. **FEFO**: al transferir, sale primero el lote que vence primero.
6. **Todo o nada**: si falta stock de algún ítem, no se descuenta nada.
7. Un lote vencido **nunca** se transfiere.
8. **Nada se borra**: se desactiva (baja lógica).
9. El stock **nunca** queda negativo, ni con dos personas transfiriendo a la vez.
10. Cada cambio de stock deja un **movimiento** con quién, cuándo y por qué (trazabilidad).

2. Repaso de programación

Concepto	En una frase	En el proyecto
Variable y tipo	Caja con nombre; el tipo dice qué cabe.	`int cantidad`, `LocalDate vence`.
Función / método	Bloque con nombre que recibe datos y devuelve un resultado.	`lote.estaVencido(hoy)`.
Bucle	Repetir algo para cada elemento.	El bucle que recorre lotes en el FEFO.
Clase / objeto	Molde / cosa hecha con el molde.	`Lote` y cada lote real (`L2345`).
Encapsulamiento	Los datos privados solo cambian por métodos con reglas.	`Lote` no tiene setters: solo `descontar`, `ajustar`, `darDeBaja`.
Constructor	Hace que el objeto nazca válido.	`new Lote(...)` rechaza cantidad ≤ 0.
Interfaz	Contrato que otra clase implementa.	`NotificadorRecuperacion` ← `CorreoSimulado`.
Excepción	Error que corta la ejecución y sube hasta que alguien lo maneja.	Todas suben hasta `ManejadorErrores`.
Lista / conjunto / mapa	En orden / sin repetir / por clave.	Ítems repetidos se suman con `Map.merge`.
`Optional`	"Puede haber o no".	`findById(id).orElseThrow(...)` → 404.

> **Ojo:** en Java los textos se comparan con `a.equals(b)`, no con `==`. Los comentarios del proyecto explican el **por qué**, no el **qué**.

3. Cómo funciona la web

- **Cliente** (el navegador) pide; **servidor** (el backend) responde.
- **URL**: `http://localhost:8080/api/medicamentos/5?activo=true` = protocolo, host, puerto, ruta y parámetros.
- **HTTP** no tiene memoria (*stateless*): cada pedido trae el token que dice quién sos.
- **JSON** es el formato de los datos. **Jackson** convierte Java ↔ JSON.
- **API REST**: cada recurso tiene una URL y se opera con los métodos HTTP. Cada método + ruta es un **endpoint**.

Método	Para qué	Ejemplo
GET	Leer.	`GET /api/medicamentos`
POST	Crear o ejecutar una acción.	`POST /api/transferencias`
PUT	Reemplazar datos.	`PUT /api/areas/3`
PATCH	Cambiar una parte.	`PATCH /api/areas/3/estado`
DELETE	Borrar.	**No se usa**: bajas lógicas.

Código	Cuándo
200 / 201 / 204	Bien / creado (con `Location`) / bien sin cuerpo.
400	Datos inválidos.
401	**"No sé quién sos"**: sin token, token alterado o vencido.
403	**"Sé quién sos, pero no podés"**: falta el permiso.
404	No existe.
409	Choca con una regla (stock insuficiente, código repetido).
500	Error inesperado (solo se muestra un `errorId`).

Los errores salen en formato **Problem Details (RFC 9457)** con un `codigo` propio: `detail` es el mensaje para la persona y `codigo` es para el programa (por ejemplo, `STOCK_INSUFICIENTE`).

El frontend es una **SPA**: se carga una vez y cambia de pantalla sin recargar. Para evitar CORS, un **proxy** (Vite en desarrollo, nginx en producción) reenvía `/api` al backend, así el navegador ve un solo origen. En el hospital hay que poner **HTTPS** en ese proxy.

4. Bases de datos y SQL

- **Tabla**: filas (registros) y columnas con tipo fijo.
- **Clave primaria (PK)**: identifica cada fila; en todas las tablas es un `id` numérico (**clave sustituta**). Las claves naturales (CUIT, código) se protegen con `UNIQUE`.
- **Clave foránea (FK)**: apunta a otra tabla (`lote.medicamento_id`). La base no deja cargar una FK que no existe (**integridad referencial**).
- **Relaciones**: 1:N (un medicamento, muchos lotes) y N:M con tabla intermedia (`detalle_transferencia` entre transferencia y lote).
- **Restricciones**: `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`. El `CHECK (cantidad_actual >= 0)` es la **última barrera** contra el stock negativo.
- **SQL**: `select`, `insert`, `update`, `join`, `group by` + `sum`. El stock total se calcula con `SUM` sobre los lotes no vencidos.
- **Parámetros**: nunca se pega texto del usuario en el SQL (evita **inyección SQL**). El `sort` se valida contra una lista blanca.
- **Índices**: buscan rápido sin leer toda la tabla.

Normalización (3FN)

Cada dato depende **"de la clave, de toda la clave y de nada más que la clave"**. Por eso no se guardan el nombre del medicamento en el lote, el stock total, el estado del lote, el número `TR-000042` ni el total de una transferencia: todo se obtiene con `JOIN`, `SUM` o se calcula.

Única excepción justificada: `movimiento_stock.saldo_resultante`. Es un dato histórico que nunca se modifica, así que no puede contradecirse, y sirve para auditoría.

Transacciones, concurrencia y migraciones

- **Transacción (ACID)**: todo o nada. Cada método con `@Transactional` es una; si lanza una excepción, se revierte entera.
- **Condición de carrera**: dos personas leen 100, piden 60 cada una y se entregan 120.
- **Bloqueo pesimista** (`SELECT … FOR UPDATE`): reserva las filas; la segunda espera.
- **Deadlock**: dos se esperan mutuamente. Se evita bloqueando siempre **en el mismo orden** (por `id`).
- **Flyway** aplica las migraciones `V1__`, `V2__`, `V3__`… en orden. **Una migración aplicada nunca se edita.** Hibernate solo valida (`ddl-auto: validate`).

Tabla	Guarda
`usuario`	Quién puede entrar y con qué rol.
`proveedor`	Quién nos vende.
`area`	A quién le entregamos.
`medicamento`	El catálogo.
`lote`	Cada partida recibida, con vencimiento y cantidad.
`transferencia`	Cada entrega a un área (cabecera).
`detalle_transferencia`	De qué lote salió cada parte de una entrega.
`movimiento_stock`	Cada cambio de cantidad de cada lote (auditoría).
`recuperacion_password`	Códigos para recuperar la contraseña.

5. Java

- **Java 21** (LTS). Un archivo `.java` = una clase pública con el mismo nombre, dentro de un paquete (`ar.edu.utn.farmacia.area`).
- `int` no puede ser `null`, `Integer` sí. Los ids son `Long` porque valen `null` hasta que la base los asigna.
- **Fechas** (`java.time`): `LocalDate` para vencimientos, `OffsetDateTime` para instantes. "Hoy" siempre es `LocalDate.now(clock)`, con un `Clock` en la zona de Buenos Aires.
- **Records**: clases de datos inmutables. Todos los DTO (`...Request`, `...Response`) son records, con un método `desde(entidad)`. El **constructor compacto** normaliza (por ejemplo, el código en mayúsculas).
- **Enums**: listas cerradas. `Rol` es el único lugar donde se define qué permisos tiene cada rol.
- **Streams**: cadena de `filter`, `map`, `sorted`, `groupingBy`. El FEFO ordena con un `Comparator` (vencimiento → fecha de ingreso → id).
- **Excepciones del dominio** (todas unchecked):

Excepción	Respuesta
`RecursoNoEncontradoException`	404 `NO_ENCONTRADO`
`ReglaNegocioException(codigo, mensaje)`	409 con ese código
`StockInsuficienteException(faltantes)`	409 `STOCK_INSUFICIENTE` + faltantes
`ErrorDeCampoException(campo, mensaje)`	400 `VALIDACION`

- **Lombok**: solo `@Getter`, `@Setter` selectivo y `@RequiredArgsConstructor`. **Nunca** `@Data` (puede exponer el hash o romper JPA).

> **Cómo leer una clase:** paquete → anotaciones de la clase → atributos `private final` (sus dependencias) → métodos públicos → comentarios.

6. Spring Boot

Spring Boot 4.1 (la 3.5 dejó de tener soporte gratuito en junio de 2026) arranca el servidor web, se conecta a la base, maneja la seguridad y convierte JSON. Nosotros escribimos solo las reglas de la farmacia.

Inyección de dependencias

Las clases no crean sus dependencias: las **piden en el constructor** y Spring se las pasa. Así cada parte se puede probar sola, reemplazando las demás por simulaciones. Regla del proyecto: inyección **por constructor**, nunca `@Autowired` en un atributo.

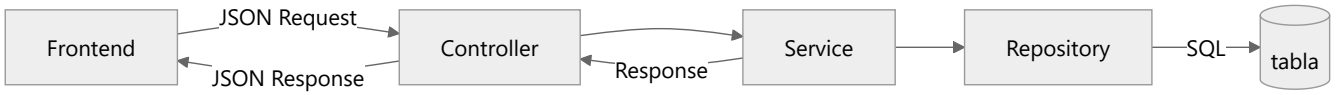
Anotación	Capa
`@RestController`	Recibe HTTP y devuelve JSON. **Sin lógica.**
`@Service`	Reglas de negocio, con `@Transactional`.
Repositorio (`JpaRepository`)	Acceso a datos; Spring lo implementa solo.
`@Configuration` + `@Bean`	Fabrica beans a mano (`Clock`, seguridad).

Las piezas principales

- **Controladores**: `@GetMapping`, `@PostMapping`..., `@PathVariable`, `@RequestParam`, `@RequestBody`, `@Valid` (si falla → 400) y `@PreAuthorize` (si falta el permiso → 403).
- **Servicios**: `@Transactional` revierte todo si hay una excepción. Dentro de la transacción, un objeto modificado se guarda solo, sin `save`.
- **JPA / Hibernate / Spring Data**: `@Entity`, `@ManyToOne`... mapean clases a tablas. Los repositorios entienden nombres de método (`existsByNombreIgnoreCase`) y admiten `@Query` y `@Lock(PESSIMISTIC_WRITE)`.
- **Validación**: `@NotBlank`, `@Size`, `@Positive`, `@Email` y `@Cuit` (propia del proyecto).
- **Errores centralizados**: `ManejadorErrores` (`@RestControllerAdvice`) convierte toda excepción en ProblemDetail. Un 500 solo muestra un `errorId`; el detalle va al log.
- **Configuración**: `application.yml` lee variables de entorno (`${JWT_SECRET:}`). Si `JWT_SECRET` falta o tiene menos de 32 bytes, **la aplicación no arranca**.
- **Perfiles**: `dev`, `demo`, `prod`, `test`. `demo` + `prod` juntos no arrancan.

7. Recorrido del backend

Organización **por funcionalidad**: `auth/`, `usuario/`, `proveedor/`, `area/`, `medicamento/`, `lote/`, `transferencia/`, `alerta/`, `common/` y `demo/`. Cada carpeta repite el mismo patrón:



Viaje de un pedido (crear un área): filtros de seguridad validan el token (401) → el controlador chequea el permiso (403) y valida el cuerpo (400) → el servicio aplica la regla de nombre único (409) → el repositorio hace el `INSERT` → responde **201** con `Location`.

Ingreso de mercadería: valida todo **antes** de modificar (medicamento y proveedor activos, vencimiento no pasado, lote no repetido). El usuario sale del **token**, nunca del JSON. Crea el lote y su movimiento `INGRESO` en la misma transacción.

Transferencia (el más importante): suma ítems repetidos → valida área y medicamentos activos → bloquea los lotes (`FOR UPDATE` ordenado por id) → ordena FEFO → calcula **todos** los faltantes (si hay, 409 y rollback) → descuenta lote por lote creando detalle y movimiento → responde el comprobante (201).

Otras piezas: `common/` tiene el manejo de errores, el reloj, la lista blanca de orden y los reportes (un solo modelo `Reporte`, tres formatos: CSV, XLSX y PDF). `alerta/` calcula las alertas y el panel en cada pedido, sin guardar nada. `demo/` carga datos de ejemplo usando los propios servicios, solo con el perfil `demo`.

8. Seguridad

- **Autenticación** = ¿quién sos? **Autorización** = ¿qué podés hacer?
- **Contraseñas**: se guarda un **hash BCrypt** (costo 12, con sal por usuario), nunca la contraseña. Un hash no se revierte (no es encriptar). Ningún DTO incluye el hash.
- **Login**: mismo mensaje para todo error ("Usuario o contraseña incorrectos") y mismo tiempo de respuesta, para no revelar qué usuarios existen. **No hay registro público.**
- **JWT**: cabecera + datos + firma HMAC-SHA256 con `JWT_SECRET`. El contenido **no está cifrado**, por eso solo lleva el id del usuario, emisor y vencimiento (8 h). Si alguien lo altera, la firma no coincide → 401.
- **En cada pedido**: se verifica firma, vencimiento y emisor; se busca el usuario en la base (si no existe o está inactivo → 401) y se cargan los **permisos de su rol actual**. Por eso una baja o un cambio de rol tiene efecto inmediato.
- **Permisos y no roles** en el código (`hasAuthority('STOCK_INGRESAR')`): un rol nuevo solo toca el enum `Rol`.
- **Recuperación de contraseña**: código de 6 dígitos con `SecureRandom`, guardado como hash, vence en 15 minutos, máximo 5 intentos. El contador se guarda aunque la operación falle y con bloqueo pesimista. El envío hoy es simulado.

Ataque	Defensa
Inyección SQL	Consultas parametrizadas + lista blanca para el orden.
XSS	React escapa el texto, ESLint prohíbe `innerHTML`, CSP en nginx y una prueba propia.
Fórmulas en CSV	Se antepone `'` a valores que empiezan con `=`, `+`, `-`, `@`.

Ataque	Defensa
CSRF	No aplica: la API no usa cookies, el token va en `Authorization`.
CORS	Mismo origen detrás del proxy; nunca `*`.
Fuerza bruta	BCrypt hace lento cada intento; límite de intentos queda como mejora.

El token se guarda en **`sessionStorage`**: se borra al cerrar la pestaña (PC compartidas). Al recargar, el frontend pide `/api/auth/me` para tener los permisos frescos.

Los 8 problemas del prototipo, resueltos

#	Problema	Solución
1	Identidad en una cabecera falsificable	Identidad solo del JWT; permisos de la base.
2	Clave del JWT en el código	Variable de entorno de 32 bytes o más.
3	Stock negativo	Validación + entidad + CHECK + bloqueo.
4	Hash expuesto	Solo DTO, sin hash.
5	Registro libre	Solo ADMIN crea usuarios.
6	Fechas como texto	Tipos de fecha reales + `Clock`.
7	Borrado en cascada	Bajas lógicas, FK sin cascada.
8	Errores genéricos, sin tests	ProblemDetail con código; pruebas en `verify`.

9. Reglas de negocio: FEFO, stock y concurrencia

Estados de un lote (hoy 12/11/2026, por vencer = 30 días)

Vence	Cantidad	Estado
11/11/2026	20	**VENCIDO** (fecha < hoy)
12/11/2026	50	**POR_VENCER** (vence hoy: todavía se usa)
12/12/2026	80	**POR_VENCER** (justo 30 días, incluido)
13/12/2026	100	**VIGENTE** (31 días)
cualquiera	0	**AGOTADO** (se mira primero)

El estado **no se guarda**: cambia con el paso del tiempo, así que se calcula.

FEFO con un ejemplo

FEFO (primero vence, primero sale) y no FIFO: FIFO entregaría el lote más viejo aunque venza más tarde y dejaría vencer el otro. Desempates: fecha de ingreso y después id, para que el resultado sea siempre el mismo.

Guardia pide **150 Paracetamol**. Lotes: L2290 (vencido, se ignora), L2301 (dic, 100), L2340 (mar, 410), L2350 (mar, 500, ingresó después).

Lote	Toma	Queda
L2301	100	0

Lote	Toma	Queda
L2340	50	360

Se crean dos renglones de detalle y dos movimientos `EGRESO`.

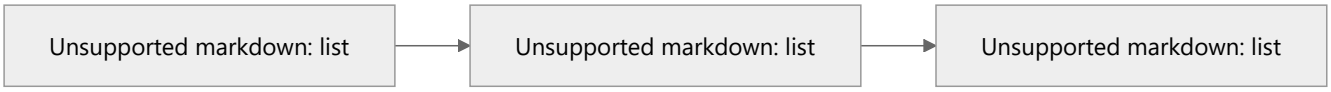
Todo o nada

Primero se calculan **todos** los faltantes y recién después se toca algo. Si falta Amoxicilina y Ceftriaxona, el 409 informa **los dos** y no se descuenta nada, ni siquiera lo que sí alcanzaba. El frontend marca cada línea ("Disponible: 12 (pediste 50)") y conserva lo cargado.

Concurrencia

Los lotes se bloquean con `SELECT … FOR UPDATE` **en una sola consulta ordenada por id**. El bloqueo evita que dos transferencias usen el mismo stock (una recibe 201, la otra 409) y el orden fijo evita el deadlock. Lo demuestra `TransferenciaConcurrenciaIT` contra PostgreSQL real.

Stock nunca negativo: tres barreras



Si una capa falla, la siguiente lo detiene (**defensa en profundidad**).

Trazabilidad, bajas y ajustes

- **Suma de los movimientos de un lote = cantidad actual del lote.** `FlujoCompletoIT` lo verifica.
- **Baja por vencimiento**: solo lotes vencidos y con stock; deja el lote en 0 (`LOTE_NO_VENCIDO`, `LOTE_SIN_STOCK` si no corresponde).
- **Ajuste**: nueva cantidad entre 0 y la inicial, distinta de la actual, con motivo; registra la diferencia.
- **Alertas** (calculadas, nunca guardadas): vencido, por vencer, sin stock y stock bajo (el más crítico primero: stock ÷ mínimo).

Otras reglas

Regla	Por qué
Medicamento con stock no vencido no se desactiva.	Quedaría mercadería invisible.
Proveedor con stock sí se desactiva.	Su mercadería sigue disponible.
Área inactiva no recibe transferencias.	El servicio cerró.
Nadie se desactiva a sí mismo; siempre queda un ADMIN activo.	Evita quedarse afuera.
Las transferencias no se editan ni se anulan.	El historial tiene que ser confiable; se corrige con ajustes.
CUIT con dígito verificador (pesos 5-4-3-2-7-6-5-4-3-2, módulo 11).	Evita errores de tipeo.

La hora de Argentina: los contenedores corren en UTC. A las 21:00 en Buenos Aires ya es mañana en UTC y un lote que vence hoy aparecería vencido. Por eso "hoy" sale siempre de un `Clock` con zona `America/Argentina/Buenos_Aires`. En el frontend, las fechas sin hora se tratan como texto para no caer en la trampa de `new Date('2026-11-30')`, que en Argentina mostraría el 29/11.

10. JavaScript y TypeScript

- **TypeScript** = JavaScript con tipos que se verifican al compilar, en modo **estricto**.
- Los tipos de los datos del backend se **generan** desde su contrato OpenAPI (`src/api/esquema.ts`): si el backend cambia un campo, el frontend deja de compilar.
- `const` (lo normal) y `let`; `type` y uniones (`'SIN_STOCK' | 'STOCK_BAJO' | 'OK'`).
- Listas: `map`, `filter`, `find`, `some`, `reduce`. Copias con `...` (**inmutabilidad**).
- `?.` (acceso seguro), `??` (valor por defecto), template strings.
- **Promesas y `async` / `await`** para esperar al servidor sin congelar la pantalla. El único `fetch` de la aplicación está en `src/api/cliente.ts`.
- Herramientas: Node, npm, Vite, tsc, ESLint, Prettier, knip y Vitest.

11. React

> **Idea central:** pantalla = función(datos). Cambian los datos → React redibuja lo necesario.

Concepto	En una frase
Componente	Función que devuelve un pedazo de pantalla (`<Insignia texto="Vencido" tono="peligro" />`).
JSX	HTML dentro de TypeScript; lo que va entre `{}` se muestra **como texto** (protege de XSS).
Props	Datos que un componente recibe de su padre.
Estado (`useState`)	Memoria del componente; se cambia siempre con un valor **nuevo**.
Hooks	`useEffect`, `useRef`, `useContext`, `useId` y propios (`useFormulario`, `useFiltrosUrl`, `useDebounce`, `useDescarga`).
Contexto	Datos para toda la app: `useSesion()` (usuario, `puede(permiso)`) y `useAvisos()`.
TanStack Query	Pide, guarda en caché (30 s) e **invalida** los datos del servidor.
React Router	Pantallas según la URL sin recargar; `RutaProtegida` exige sesión y permiso.

Después de un cambio de stock se invalidan medicamentos, lotes, movimientos, alertas, resumen, transferencias y proveedores, para que todas las vistas se actualicen.

12. Recorrido del frontend

`frontend/src/` tiene `api/` (la única puerta al backend), `auth/` (sesión), `components/`, `hooks/`, `pages/` (una carpeta por pantalla), `utils/` (funciones puras) y `styles/`.

`cliente.ts` arma cada pedido con el token, pone un máximo de 15 segundos y traduce los errores:

Respuesta	Qué hace el frontend
401	Borra la sesión y lleva al ingreso ("Tu sesión expiró").
403	"No tenés permiso para esta acción."
400 / 404 / 409	Muestra el `detail` del backend.
5xx	"Ocurrió un error en el servidor. Código: {errorId}."
Sin red / 15 s	"No se pudo conectar…" / "El servidor tardó demasiado…"

Nueva transferencia (la pantalla más compleja): borrador en `sessionStorage`, suma de líneas repetidas, validación de cada cantidad, modal de revisión, bloqueo del doble clic con `useRef` y, ante un 409, marca las líneas con faltante sin perder lo cargado.

Estilos: CSS propio con variables en `tokens.css` y nombres BEM (`.boton--primario`), sin Bootstrap ni Tailwind.

Accesibilidad: contraste WCAG AA, todo usable con teclado, etiquetas en todos los campos, estados con palabra (no solo color), `<dialog>` nativo, `role="status"` / `"alert"` en los avisos, `prefers-reduced-motion` y verificación con `jsx-a11y` y axe.

13. Docker y entornos

Concepto	Analogía
Imagen	La receta.
Contenedor	La torta hecha con la receta.
Dockerfile	La receta escrita.
Volumen	Disco externo: los datos de la base sobreviven al contenedor.
Compose	Levanta varios contenedores juntos (`db`, `api`, `web`).

- Dentro de Compose los servicios se encuentran **por nombre** (`db:5432`).
- Dockerfile **multi-etapa**: compila con el JDK y ejecuta solo con el JRE, **sin root**. El `-DskipTests` del Dockerfile es la única excepción a no saltear pruebas.
- **Variables de entorno**: `.env.example` (plantilla, va a git) y `.env` (valores reales, **no** va a git).
- Puertos publicados solo en `127.0.0.1`.

Levantar todo: copiar `.env.example` a `.env`, completar `DB_PASSWORD`, `JWT_SECRET` y `ADMIN_PASSWORD`, y correr `docker compose up --build`. `docker compose down -v` borra también los datos.

CI: GitHub Actions corre `./mvnw verify` y los chequeos del frontend en cada push.

14. Pruebas

Estructura de toda prueba: **preparar, actuar, verificar**. El nombre dice qué comportamiento se espera.

Tipo	Qué prueba	Ejemplo
Unitaria	Una pieza sola, con simulaciones (mocks).	`LoteTest`, `TransferenciaServiceTest`, `fechas.test.ts`
Integración	Varias piezas con PostgreSQL real (Testcontainers).	`FlujoCompletoIT`, `SeguridadIT`

Tipo	Qué prueba	Ejemplo
Punta a punta	El sistema entero en Chrome, con axe.	`e2e/capturas.spec.ts`

- **Backend**: JUnit 5, AssertJ, Mockito, MockMvc, Testcontainers y JaCoCo. `*Test` = unitarias, `*IT` = integración. `./mvnw verify` corre todo.
- **Frontend**: Vitest, Testing Library (busca por rol y texto visible, como una persona), jsdom, user-event, Playwright y axe-core.
- **Reloj fijo** en las pruebas: "hoy" es siempre la misma fecha, así los bordes de vencimiento dan igual cualquier día.
- ¿Por qué PostgreSQL real y no H2? Porque se prueban los `CHECK`, el `FOR UPDATE` y el SQL propio de PostgreSQL.
- **TDD** (rojo → verde → refactorizar) en la lógica crítica: FEFO, alertas, CUIT y lote.

Si te preguntan…	La prueba es…
¿Funciona el FEFO?	`TransferenciaServiceTest` y `FlujoCompletoIT`.
¿Y con dos personas a la vez?	`TransferenciaConcurrenciaIT`.
¿No se ve el hash?	`SeguridadIT.ningunaRespuestaExponeLaContrasenaNiElHash`.
¿Y los bordes de vencimiento?	`AlertaServiceTest`.
¿Y si falta la clave del JWT?	`ConfiguracionTest`.
¿Los movimientos cierran con el stock?	`FlujoCompletoIT`.
¿El CUIT se valida igual en front y back?	`CuitValidatorTest` y `cuit.test.ts`.
¿Es accesible?	`capturas.spec.ts` con axe.

15. Git y herramientas

- **Git**: repositorio, commit, rama, merge, remoto, push/pull y pull request. `.gitignore` deja afuera `.env`, `node_modules` y `target`.
- **Conventional Commits** en español: `feat(transferencia): descontar stock con criterio FEFO`. Commits chicos y coherentes.
- **Maven** (`pom.xml`, wrapper `mvnw`): `verify` = formato + unitarias + integración. El BOM de Spring Boot elige versiones compatibles.
- **npm** (`package.json`, `package-lock.json`): `dev`, `build`, `lint`, `typecheck`, `test`, `api:tipos`, `mock`, `capturas`.
- **Calidad automática**: Spotless (Java), ESLint sin advertencias, Prettier, TypeScript estricto y knip.
- **OpenAPI y Swagger UI**: documentación de la API generada desde los controladores (`/v3/api-docs`, `/swagger-ui.html`).
- **IA**: se usó un asistente de programación, con cada parte especificada antes, revisada y verificada con pruebas, y las decisiones registradas con su justificación.

16. Checklist antes de la defensa

- [ ] Puedo explicar el sistema en un minuto.
- [ ] Puedo dibujar la arquitectura (navegador → frontend → backend → base).
- [ ] Puedo dibujar el diagrama ER y justificar la 3FN.

- [ ] Puedo explicar FEFO con un ejemplo numérico.
- [ ] Puedo explicar la concurrencia, el bloqueo y el deadlock.
- [ ] Puedo explicar login, BCrypt, JWT, roles, 401 y 403.
- [ ] Sé qué prueba demuestra cada problema del prototipo resuelto.
- [ ] Levanté el sistema con Docker y practiqué la demo.
- [ ] Leí el guion de la defensa.